

Generazione del report PDF Avviso di fattura

Documento tecnico-didattico dell'implementazione (fase B3b)
ICM Fatturazioni — Blazor Server / .NET 10 / PDFsharp-MigraDoc

Come funziona, passo per passo, con riferimento al codice sorgente

1. Obiettivo e contesto

L'**avviso di fattura** (o *avviso di parcella*) e' il documento che lo studio consegna al cliente prima di emettere la fattura vera e propria: riepiloga le prestazioni, applica la cascata fiscale (cassa previdenza C.N.P.A.I.A., IVA, ritenuta d'acconto, spese anticipate ex art. 15) e riporta le condizioni di pagamento. Non ha valore fiscale ("Non costituisce fattura"), ma deve essere stampabile in un PDF ordinato e fedele al modulo storico dello studio.

L'obiettivo della fase **B3b** era generare quel PDF a partire da un avviso gia' emesso e salvato a database, riproducendo fedelmente il modello fornito (`docs/AvvisoDiFattura.pdf`): intestazione dello studio e del cliente, barra del titolo, riga di pagamento e banca, righe del documento, cascata dei totali e nota legale.

Questo documento spiega, in modo didattico, **tutti i passaggi** necessari e come sono stati realizzati, con i riferimenti ai file e al codice sorgente.

1.1 Il principio guida: separare i dati dal disegno

L'idea centrale — ripresa dal gemello ICMVerbalì — e' che la generazione di un PDF sia la composizione di due responsabilita' distinte:

- **Raccolta dati** (asincrona, tocca il database): un servizio legge tutto cio' che serve e lo impacchetta in un oggetto immutabile.
- **Disegno** (sincrono, nessun accesso ai dati): una classe di rendering trasforma quell'oggetto in byte PDF.

Il vantaggio e' che il disegno diventa una funzione pura `dati → byte[]`: facile da testare (basta costruire i dati a mano) e privo di dipendenze da database, rete o sessione HTTP.

2. La catena completa in breve

Dal click dell'utente al PDF nel browser attraversiamo cinque componenti, ciascuno con una sola responsabilita':

#	Componente	File	Responsabilita'
1	Pulsante UI	<code>AvvisiFattura.razor</code>	Apri l'endpoint in una nuova scheda
2	Endpoint HTTP	<code>Program.cs</code>	Serve i byte come <code>application/pdf</code>
3	Servizio	<code>AvvisoPdfService.cs</code>	Carica i dati e ricalcola i totali
4	DTO immutabile	<code>AvvisoPdfData</code>	Trasporta i dati gia' risolti
5	Rendering	<code>AvvisoPdfDocument.cs</code>	Disegna il PDF con MigraDoc

Il flusso e' unidirezionale: la UI non conosce il servizio PDF, e il servizio PDF non conosce MigraDoc oltre alla classe di rendering. Ognuno dipende solo dal livello immediatamente sotto.

Regola architetturale rispettata: il servizio PDF non chiama mai direttamente i Repository, ma passa sempre dai Manager (UI → Manager → Repository), come impone l'architettura a strati del progetto.

3. Passaggio 0 — La libreria di generazione

Per produrre il PDF si usa **PDFsharp-MigraDoc 6.2.0**, la stessa gia' adottata da ICMVerbalì per i verbali. MigraDoc lavora a un livello alto: si costruisce un documento logico (sezioni, tabelle, paragrafi, stili) e la libreria si occupa dell'impaginazione e degli a-capo, senza dover posizionare il testo a coordinate.

```
<PackageReference Include="PDFsharp-MigraDoc" Version="6.2.0" />
```

E' l'unico pacchetto aggiunto (licenza MIT, già approvato). Non servono font esterni: si usano i **font di sistema di Windows** (vedi §6.1), quindi nessun file embedded.

4. Passaggio 1 — I dati che mancavano: lo studio emittente

Un PDF di avviso ha in cima i dati del **cedente** (lo studio): ragione sociale, indirizzo, recapiti, codice fiscale e partita IVA. Analizzando il sistema e' emerso che **questi dati non esistevano da nessuna parte**: ne' tabella, ne' entita', ne' configurazione. Nel gestionale Access di origine stavano nella tabella `TAB_Azienda`.

La scelta e' stata ricrearli come tabella vera, coerente con le convenzioni del progetto (chiave GUID v7, soft-delete), e **completa dei campi utili alla futura fattura elettronica** (REA, CCIAA, capitale sociale, regime fiscale...), così da non dover rifare la migration in Fase D.

La migration (repository ICMVerbali)

Lo schema del database e' di proprieta' di ICMVerbali (DB unificato): la tabella nasce come migration numerata li'. Estratto essenziale:

Migrations/060_FattAzienda.sql (estratto)

```
IF OBJECT_ID(N'fatt.Azienda', N'U') IS NULL
BEGIN
    CREATE TABLE fatt.Azienda
    (
        IdAzienda        UNIQUEIDENTIFIER NOT NULL CONSTRAINT PK_Azienda PRIMARY KEY,
        Azienda          NVARCHAR(50)    NOT NULL,      -- nome breve/alias
        RagioneSociale   NVARCHAR(100)   NOT NULL,
        PIVA             NVARCHAR(11)     NULL,
        CodiceFiscale    NVARCHAR(16)     NULL,
        IndirizzoVia     NVARCHAR(100)    NULL,
        IndirizzoCivico  NVARCHAR(10)     NULL,
        IndirizzoComune  NVARCHAR(100)    NULL,
        IndirizzoProvincia NVARCHAR(50)   NULL,
        IndirizzoCAP     NVARCHAR(5)      NULL,
        Telefono         NVARCHAR(50)     NULL,
        Telefax          NVARCHAR(50)     NULL,
        Email            NVARCHAR(100)    NULL,
        PEC              NVARCHAR(100)    NULL,
        -- ... campi Fattura Elettronica (REA, CCIAA, RegimeFiscale, ...) ...
        IsAttivo         BIT NOT NULL CONSTRAINT DF_Azienda_IsAttivo DEFAULT 1
    );
END
```

Un **seed idempotente** inserisce la singola riga reale dello studio, ma solo se la tabella e' vuota, così un secondo avvio non duplica ne' sovrascrive modifiche fatte a mano:

Migrations/060_FattAzienda.sql (seed)

```
IF NOT EXISTS (SELECT 1 FROM fatt.Azienda)
BEGIN
    INSERT INTO fatt.Azienda (IdAzienda, Azienda, RagioneSociale, ...)
    VALUES ('0197f060-...-000000000001', N'Bortolaso Vantini',
            N'Vantini Oliosio Nardon Architetti', ...);
END
```

Entita', repository e manager (lato ICMFatturazioni)

In C# si aggiunge il consueto trio a strati. L'entita' e' un POCO; da notare la proprieta' `NomeBreve`, mappata sulla colonna legacy `Azienda` (un membro non puo' chiamarsi come la classe che lo contiene):

Entities/Azienda.cs (estratto)

```
public sealed class Azienda
{
    public Guid IdAzienda { get; set; }
    public required string NomeBreve { get; init; } // colonna legacy "Azienda"
    public required string RagioneSociale { get; init; }
    public string? PIVA { get; init; }
    public string? CodiceFiscale { get; init; }
    public string? IndirizzoVia { get; init; }
    // ... indirizzo, recapiti, Email, PEC, campi FE ...
    public bool IsAttivo { get; init; } = true;
}
```

Il repository legge "l'azienda corrente" (sistema mono-studio: la prima riga attiva). La colonna `Azienda` viene rinominata in `NomeBreve` direttamente nella query:

Repositories/AziendaRepository.cs (estratto)

```
private const string SqlSelectCorrente = """
    SELECT TOP (1)
        IdAzienda, Azienda AS NomeBreve, RagioneSociale, PIVA, CodiceFiscale,
        IndirizzoVia, IndirizzoCivico, IndirizzoComune, ... , IsAttivo
    FROM fatt.Azienda
    WHERE IsAttivo = 1
    ORDER BY RagioneSociale;
""";

public async Task<Azienda?> GetAziendaAsync(CancellationTokentoken ct = default)
{
    using var conn = await _connectionFactory.CreateOpenConnectionAsync(ct);
    var cmd = new CommandDefinition(SqlSelectCorrente, cancellationTokentoken: ct);
    return await conn.QuerySingleOrDefaultAsync<Azienda>(cmd);
}
```

Il manager (`AziendaManager`) e', per ora, una semplice delega di lettura: non c'e' ancora scrittura sui dati dello studio (una pagina di gestione e' rimandata).

5. Passaggio 2 — Le spese anticipate dell'avviso

La cascata fiscale ha bisogno della somma delle **spese anticipate** (art. 15) collegate all'avviso. Serviva quindi una lettura "per avviso" che prima non c'era; e' stata aggiunta al repository e al manager delle spese:

Repositories/SpesaAnticipataRepository.cs (aggiunta)

```
// Spese riaddebitate in uno specifico avviso (attive), per la cascata art.15.
private const string SqlSelectByAvviso =
    SqlSelectBase + " WHERE IdAvviso = @IdAvviso AND IsAttivo = 1 ORDER BY Data ASC;";

public async Task<IReadOnlyList<SpesaAnticipata>> GetByAvvisoAsync(
    Guid idAvviso, CancellationTokentoken ct = default) { /* Dapper query */ }
```

6. Passaggio 3 — Il servizio che raccoglie i dati

`AvvisoPdfService` e' il direttore d'orchestra. Riceve l'id dell'avviso e produce i byte, coordinando sette Manager. I passi, nell'ordine:

- **Carica testata e righe** dell'avviso; se non esiste o e' annullato lancia un'eccezione tipizzata (diventera' un 404).
- **Legge in parallelo** studio, cliente, attivita' e spese collegate (letture indipendenti).
- **Risolve pagamento e banca**: sulla testata ci sono solo gli id, quindi si recuperano le descrizioni.
- **Ricalcola la cascata fiscale** dagli snapshot congelati sull'avviso.

- **Impacchetta tutto** in un DTO immutabile e delega il disegno.

Services/AvvisoPdfService.cs (metodo principale)

```
public async Task<byte[]> GeneraAsync(Guid idAvviso, CancellationToken ct = default)
{
    var dettaglio = await _avvisi.GetDettaglioAsync(idAvviso, ct);
    if (dettaglio is null || !dettaglio.Testata.IsAttivo)
        throw new AvvisoPdfNonTrovatoException(idAvviso);

    var testata = dettaglio.Testata;

    // Letture indipendenti in parallelo (una sola richiesta HTTP).
    var studioT = _azienda.GetAziendaAsync(ct);
    var clienteT = _anagrafiche.GetByIdAsync(testata.IdAnagrafica, ct);
    var attivT = _attivita.GetByIdAsync(testata.IdAttivita, ct);
    var speseT = _spese.ElencoPerAvvisoAsync(idAvviso, ct);
    await Task.WhenAll(studioT, clienteT, attivT, speseT);

    var studio = studioT.Result ?? throw new AvvisoPdfDatiMancantiException("...");
    var cliente = clienteT.Result ?? throw new AvvisoPdfDatiMancantiException("...");

    // Pagamento/banca: sulla testata ci sono solo gli id -> lookup delle descrizioni.
    string? descrizionePagamento = null;
    if (testata.IdCodicePagamento is { } idPag)
        descrizionePagamento = (await _pagamenti.GetByIdAsync(idPag, ct)).DescrPag;

    string? descrizioneBanca = null;
    if (testata.IdBancaAppoggio is { } idBanca)
        descrizioneBanca = ComponiBanca(await _banche.GetByIdAsync(idBanca, ct));

    // Cascata: imponibile = somma righe reali; spese art.15 = somma spese collegate.
    var imponibile = dettaglio.Righe.Where(r => !r.IsDescrittiva).Sum(r => r.Importo ?? 0m);
    var speseArt15 = speseT.Result.Sum(s => s.Importo);
    var calcolo = _avvisi.Calcola(testata, imponibile, speseArt15);

    var data = new AvvisoPdfData(studio, cliente, attivT.Result, testata,
        dettaglio.Righe, descrizionePagamento, descrizioneBanca, calcolo);

    return new AvvisoPdfDocument(data).Render(); // disegno sincrono
}
```

Perche' "snapshot" di pagamento e banca

Sulla testata dell'avviso pagamento e banca sono salvati come **id copiati dall'anagrafica al momento dell'emissione** (uno "scatto fotografico"). Così un avviso storico resta immutabile anche se in seguito si modifica il cliente. Conseguenza pratica: se al momento dell'emissione il cliente non aveva pagamento o banca, il PDF li mostra come "—"; per popolarli occorre completare l'anagrafica e **ri-emettere** l'avviso.

Il DTO immutabile

Il "pacchetto" che passa al rendering e' un `record` immutabile: contiene solo dati già risolti, nessun Manager, nessuna connessione. E' ciò che rende il disegno una funzione pura.

Services/AvvisoPdfService.cs

```
internal sealed record AvvisoPdfData(
    Azienda Studio,
    Anagrafica Cliente,
    Attivita? Attivita,
    AvvisoFattura Testata,
    IReadOnlyList<AvvisoFatturaRiga> Righe,
    string? DescrizionePagamento,
    string? DescrizioneBanca,
    CalcoloFiscaleRisultato Calcolo);
```

7. Passaggio 4 — Il disegno del PDF

AvvisoPdfDocument e' la classe che trasforma il DTO in byte. Espone un solo metodo pubblico, `Render()`, che costruisce il documento MigraDoc, lo impagina e lo salva in memoria:

Services/AvvisoPdfDocument.cs

```
public byte[] Render()
{
    var document = BuildDocument();
    var renderer = new PdfDocumentRenderer { Document = document };
    renderer.RenderDocument();

    using var ms = new MemoryStream();
    renderer.PdfDocument.Save(ms, closeStream: false);
    return ms.ToArray();
}
```

7.1 I font: si usano quelli di Windows

PDFsharp 6.x non trova i font di sistema senza un "resolver" esplicito. Lo si abilita una sola volta nel costruttore statico della classe; il font di base del documento e' **Arial** (tecnico, presente su ogni installazione Windows). Nessun font viene incorporato nel PDF.

Services/AvvisoPdfDocument.cs (font)

```
static AvvisoPdfDocument()
{
    // Abilita il resolver che legge i font principali da %WINDIR%\Fonts.
    GlobalFontSettings.UseWindowsFontsUnderWindows = true;
}

private static void SetupStyles(Document doc)
{
    var normal = doc.Styles[StyleNames.Normal]!;
    normal.Font.Name = "Arial";
    normal.Font.Size = 9;
}
```

7.2 La struttura della pagina

`BuildDocument()` imposta il foglio A4 con i margini, aggiunge il piè di pagina e poi richiama, in ordine, un metodo per ciascun blocco del documento. La sequenza rispecchia esattamente il modulo cartaceo:

Services/AvvisoPdfDocument.cs (BuildDocument)

```
ComposeIntestazione(section); // studio (sx) + box cliente (dx)
ComposeBarraTitolo(section); // AVVISO DI FATTURA | Data | Pagina | Valuta
ComposePagamento(section); // descrizione pagamento + banca d'appoggio
ComposeOggetto(section); // OGGETTO: ...
ComposeRighe(section); // DESCRIZIONE | IMPORTO | IVA
ComposeCascata(section); // imponibile -> cassa -> IVA -> ritenuta -> avere
ComposeNoteFinali(section); // testo spese art.15 + nota legale
```

Ogni blocco e' una tabella MigraDoc. Alcune tecniche ricorrenti:

- **Intestazione a due colonne:** una tabella senza bordi con la colonna studio a sinistra e, a destra, una cella con bordo e sfondo azzurrino per il riquadro del cliente.
- **Barra del titolo:** tabella a 4 colonne dove la cella "AVVISO DI FATTURA" occupa due righe (MergeDown), mentre a fianco stanno le etichette Data/Pagina/Valuta e i rispettivi valori.
- **Righe del documento:** le righe descrittive non hanno importo ne' IVA; le righe reali mostrano "€", l'importo allineato a destra e l'aliquota.
- **Numero di pagina:** reso con un campo dinamico (AddPageField), non un testo fisso.

7.3 La cascata fiscale

E' il cuore contabile del documento. Il servizio ha gia' calcolato tutti gli importi; il rendering li dispone in righe etichetta / € / valore / segno. La stessa funzione `RigaCascata` produce ogni riga; le voci di ritenuta e spese compaiono solo se maggiori di zero:

Services/AvvisoPdfDocument.cs (cascata)

```
RigaCascata(table, "TOTALE IMPONIBILE", c.Imponibile, segno: null, bold: true);
RigaCascata(table, $"Maggiorazione {Perc(a.AliquotaCnpaia)}% C.N.P.A.I.A.",
    c.Cassa, segno: "+", bold: true);
RigaCascata(table, "", c.ImponibilePiuCassa, segno: null, bold: false); // subtotale
RigaCascata(table, $"I.V.A. {Perc(a.AliquotaIva)}% su € {Eur(c.ImponibilePiuCassa)}",
    c.Iva, segno: "+", bold: true);
RigaCascata(table, "TOTALE", c.Totale, segno: null, bold: true);

if (c.Ritenuta > 0)
    RigaCascata(table, $"Ritenuta Acconto {Perc(a.AliquotaRitenuta)}% su € {Eur(c.Imponibile)}",
        c.Ritenuta, segno: "-", bold: true);
if (c.SpeseArt15 > 0)
    RigaCascata(table, "Spese anticipate escluse art. 15", c.SpeseArt15, segno: "+", bold: true);

RigaCascata(table, "TOTALE NOSTRO AVERE S.E. & O.", c.TotaleNostroAvere,
    segno: null, bold: true, evidenzia: true);
```

Gli importi arrivano dal servizio puro `CalcoloFiscaleAvviso` (gia' esistente dalla fase B1), che il manager richiama passando gli snapshot congelati sull'avviso. Il PDF non ricalcola nulla: si limita a stampare, garantendo coerenza con l'anteprima a video.

8. Passaggio 5 — L'endpoint HTTP

Blazor Server non puo' restituire un file binario da un componente, quindi si espone un piccolo endpoint Minimal API. Serve il PDF **inline** (il browser lo apre in anteprima) e richiede l'autenticazione; in caso di errore lo registra nel log e risponde con lo stato adeguato:

Program.cs (endpoint)

```
app.MapGet("/api/avvisi/{id:guid}/pdf", async Task<IResult> (
    Guid id, HttpContext httpContext, IAvvisoPdfService pdfService,
    IAvvisoFatturaManager avvisoManager, ILogManager logManager, CancellationToken ct) =>
{
    try
    {
        var pdf = await pdfService.GeneraAsync(id, ct);
        var dettaglio = await avvisoManager.GetDettaglioAsync(id, ct);
        var nomeFile = dettaglio is not null
            ? $"Avviso_{dettaglio.Testata.DataAvviso:yyyy-MM-dd}.pdf" : $"Avviso_{id:D}.pdf";

        httpContext.Response.Headers.ContentDisposition = $"inline; filename=\"{nomeFile}\"";
        return Results.File(pdf, "application/pdf");
    }
    catch (AvvisoPdfNonTrovatoException) { return Results.NotFound(); }
    catch (Exception ex)
    {
        await logManager.LogErrorAsync(ex, "...", "AvvisoPdf.GeneraAsync",
            entityId: id, entityType: "AvvisoFattura", cancellation_token: ct);
        return Results.Problem("Errore durante la generazione del PDF.", statusCode: 500);
    }
}).RequireAuthorization();
```

9. Passaggio 6 — Il pulsante nell'interfaccia

Nella pagina degli avvisi, ogni avviso gia' emesso ha un'icona PDF. Il click apre l'endpoint in una nuova scheda del browser (che si occupa di mostrare/scaricare il file):

Components/Pages/AvvisiFattura.razor (@code)

```
private async Task AnteprimaPdfAsync(AvvisoFattura a)
{
    try { await JS.InvokeVoidAsync("open", $"/api/avvisi/{a.IdAvviso}/pdf", "_blank"); }
    catch (Exception ex)
    {
        await LogManager.LogErrorAsync(ex, "Apertura anteprima PDF fallita.",
            "AvvisiFattura.AnteprimaPdfAsync");
        Snackbar.Add("Impossibile aprire l'anteprima PDF.", Severity.Error);
    }
}
```

10. Passaggio 7 — I test

La qualità estetica del layout non si verifica a costo ragionevole; si verifica invece che il PDF **venga effettivamente generato** e che le guardie del servizio funzionino. Poiché il disegno è una funzione pura, i test più importanti costruiscono i dati a mano e controllano che l'output inizi con la firma %PDF:

tests/.../Services/AvvisoPdfServiceTests.cs (estratto)

```
[Fact]
public void Render_ConRitenutaESpese_ProducePdf()
{
    var data = CostruisciData(applcaRitenuta: true, spese: 300m, includiDescrittiva: false);
    var pdf = new AvvisoPdfDocument(data).Render();
    AssertPdfValido(pdf); // lunghezza > 0 e primi 4 byte == "%PDF"
}

[Fact]
public async Task GeneraAsync_AvvisoInesistente_LanciaNonTrovato()
{
    var service = CostruisciService(new FakeAvvisoManager { Dettaglio = null });
    await Assert.ThrowsAsync<AvvisoPdfNonTrovatoException>(
        () => service.GeneraAsync(Guid.NewGuid()));
}
```

In tutto sono 8 nuovi test: 4 sul rendering (caso base, con ritenuta e spese, sole righe descrittive, dati parziali) e 4 sulle guardie del servizio (non trovato, annullato, azienda mancante, happy path). La suite completa è di **420 test verdi**. I test usano *fake* scritti a mano (niente librerie di mock), come da convenzione del progetto.

11. Riepilogo: dal click al PDF

Mettendo in fila i passaggi visti, ecco l'intero percorso di una richiesta:

- L'utente clicca l'icona PDF su un avviso emesso → il browser apre `/api/avvisi/{id}/pdf`.
- L'endpoint chiama `AvvisoPdfService.GeneraAsync(id)`.
- Il servizio carica testata+righe, studio, cliente, attività, spese; risolve pagamento e banca; ricalcola la cascata; costruisce `AvvisoPdfData`.
- `AvvisoPdfDocument.Render()` disegna il PDF con MigraDoc e restituisce i byte.
- L'endpoint li serve come `application/pdf inline` → il browser mostra l'anteprima.

11.1 File toccati

File	Ruolo
Migrations/060_FattAzienda.sql	Tabella + seed dati studio (repo ICMVerbal)
Entities/Azienda.cs	Entità dello studio emittente
Repositories/AziendaRepository.cs	Lettura azienda corrente (Dapper)

Managers/AziendaManager.cs	Delega di lettura
SpesaAnticipataRepository/Manager	Lettura spese per avviso (aggiunta)
Services/IAvvisoPdfService.cs	Contratto del servizio
Services/AvvisoPdfService.cs	Raccolta dati + orchestrazione
Services/AvvisoPdfDocument.cs	Rendering MigraDoc del layout
Services/AvvisoPdfExceptions.cs	Eccezioni tipizzate (404 / 500)
Program.cs	Endpoint HTTP + registrazione DI
Components/Pages/AvvisiFattura.razor	Pulsante Anteprima PDF
tests/.../AvvisoPdfServiceTests.cs	8 test (rendering + guardie)

11.2 Cosa e' stato volutamente rimandato

- **Anteprima della bozza** (prima dell'emissione): la bozza vive solo lato client, senza id; servirebbe un endpoint dedicato.
- **Scelta di pagamento e banca nel wizard**: oggi si ereditano dall'anagrafica; l'override in interfaccia e' un miglioramento futuro.
- **Pagina di gestione dei dati studio**: oggi la tabella e' in sola lettura (seed); il CRUD amministrativo verra' dopo.

ICM Fatturazioni — documento tecnico generato per la fase B3b (Anteprima PDF dell'avviso di fattura).